Assignment: Ship a Containerized App with Git & Docker

Time: 60 minutes

You'll need: Git, Docker, Python 3, a GitHub account, a Docker Hub account, a terminal

The scenario

You're building a tiny "greeter" web page that says hello. You'll aversion it with Git, work through GitHub the way a team would (branches and pull requests, even though you're alone), containerize it with Docker, and publish the image so anyone can run it without installing anything.

Suggested time per part is in brackets. Don't polish anything — the goal is to do every step once.

About writing the app: the code itself is not what's being tested. Feel free to grab a starter from GitHub, adapt an open-source example, or ask an AI coding assistant to write it for you. Spend your time on the Git and Docker steps, not on the app. Write it in Python. It only needs to do one thing: when you open it in a browser, it shows a greeting with your name. A few lines using Python's built-in http.server module is enough — no framework needed.

Two terms you'll see

- Port — a numbered "door" on a machine that a program listens on. Your app will listen on one (something like 5000 or 8080). When Docker "maps" a port, it connects a door on your machine to a door inside the container so your browser can reach the app.
- Image tag — a version label on a Docker image, written after a colon, like greeter:1.0. Same name, different tag = different version.

Part 1 — Set up the repo [5 min]

- Create a new public repository on GitHub called greeter. Don't let GitHub add any files to it.
- On your machine, make a folder, initialize it as a Git repository, and connect it to the GitHub repo.
- Add your Python greeter app. Run it locally once and open it in a browser to confirm it works.
- Add an ignore file so temp files never get committed.
- Make your first commit and push it to GitHub.

Checkpoint: Your code is visible on GitHub.

Part 2 — Work on a branch, merge through a PR [10 min]

- Create a branch for a small change: add a second line to the greeting, for example today's date or a favorite quote.

- Commit the change on the branch and push the branch (not main).
- On GitHub, open a pull request into main. Write a one-sentence description of the change.
- Review your own PR: look at the diff tab and make sure only the lines you intended changed. Merge it and delete the branch.
- Back on your machine, switch to main and pull the merged result.

Checkpoint: main has two commits locally and on GitHub, and the branch is gone.

Part 3 — Cause a merge conflict and fix it [10 min]

- From main, create a branch and change the greeting word (for example "Hello" to "Hi"). Commit it.
- Switch back to main and create a *second* branch. Change the *same line* to a different word. Commit it.
- Merge the first branch into main. This should go through cleanly.
- Now merge the second branch into main. This should conflict.
- Open the file, look at the conflict markers, decide which version you want (or combine them), remove the markers, and complete the merge.
- Delete both branches. Push main.

Checkpoint: The file has one greeting line and no leftover markers. Look at the commit graph and find the point where the branches came together.

Part 4 — Containerize it [14 min]

- On a new branch, write a Dockerfile for your app: start from the official Python image, copy your code in, and set the command that starts the app.
- Add a Docker ignore file so the .git folder and any temp files stay out of the image.
- Build the image and give it a name.
- Run a container from it in the background, mapping a port on your machine to the port the app listens on.
- Open it in your browser. Check the container's logs. Stop the container.
- Run it a second time. Notice it starts instantly and there's nothing left over from the first run.
- Commit, push, open a PR, merge it, pull.

Checkpoint: Your greeting shows up in the browser, served from inside a container.

Part 5 — Run a second container alongside it [6 min]

You don't need to change your app for this part.

- Start a second container from the official nginx image (a ready-made web server), mapped to a different port than your greeter. Open it in your browser — you'll see the default nginx welcome page.

- List running containers. You should see both your greeter and nginx running side by side, each with its own name and port.
- Look at the logs of the nginx container and find the line for the request your browser just made.
- Stop and remove the nginx container. Confirm it's gone from the list.

Checkpoint: Two unrelated containers ran at the same time on different ports, and you removed one without affecting the other.

Write down your answer to: *You never installed nginx on your machine. Where did it come from, and where is it now that the container is removed?*

Part 6 — Publish the image [10 min]

- Rebuild your greeter image with a tag that includes your Docker Hub username and a version number like 1.0.
- Log in to Docker Hub and push the image.
- Prove it's portable: go to a completely different folder with none of your source code, and run the image straight from Docker Hub.
- Change the greeting, rebuild as 1.1, push it. Run 1.0 and 1.1 at the same time on two different ports.

Checkpoint: Both versions respond side by side, and neither needed your source code to run.

Part 7 — Wrap up [5 min]

- Add a README to the repo explaining in a few lines what the app does and the two commands to build and run it with Docker.
- Commit it directly to main this time and push.
- Clean up: stop and remove all containers, then check how much disk space Docker is using and prune what you don't need.

Submission

- Link to your GitHub repo
- The output of your commit graph command, pasted as text
- Your Docker Hub image name
- Your answer to the Part 5 question